AMLD Security 3.1

Certified Security Settlement

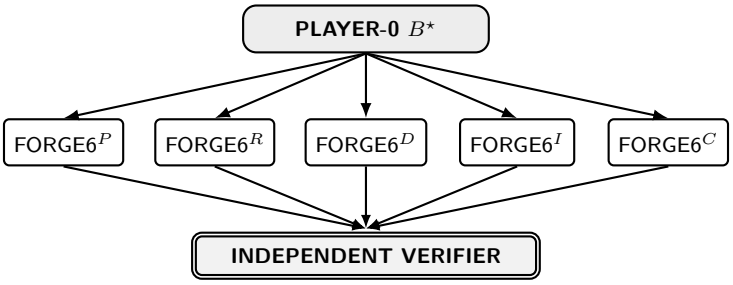
with Five FORGE-6 Federations

and Player-0 $B^\star$

A formal sequel to *AMLD Security 3:*
Eight Formal Application Sketches for Intelligent Target Settlement

Brian Tenneson

Research manuscript, ATLAS-repaired illustrated edition 0.3 · October 21, 2026



Search proposes. Verification certifies.

Status: ATLAS-repaired mathematical manuscript with explicit assumptions and proofs. No implementation, FORGE-6 run, exploit demonstration, empirical speedup, independent proof-assistant check, or universal-security result is claimed. Any role-directed investigation is conditional on a declared verifier, environment, resources and authorization.

Companion statement

Series identity. This manuscript is *AMLD Security 3.1*, a direct sequel to *AMLD Security 3*.

AMLD Security 3 established eight defensively oriented application sketches for verifier-gated target settlement [1]. Its account had four active settlement roles P, R, I, C and an explicit UNKNOWN outcome. The present sequel completes the five-role vocabulary by introducing D (Dependence), assigns to each settlement role an entire specified FORGE-6 federation, and installs a distinct global Player-0 $B^\star$. It does not replace the older propositions. It develops the additional obligations that follow from this larger, nested architecture.

In this sequel "five FORGE-6 federations" has a literal interpretation: $\mathsf{F}^P, \mathsf{F}^R, \mathsf{F}^D, \mathsf{F}^I, \mathsf{F}^C$ are five role-local FORGE-6 instances. Each has one internal coordinator B_0^r and five mathematical ATPs $B_1^r, \ldots, B_5^r$. Thus there are 25 mathematical search ATPs, five internal coordinators, and one global Player-0 $B^\star$: 31 distinguished components. This count describes an architecture, not a claim that 31 executable processes were launched.

Abstract

A verifier-gated architecture can insulate security claims from speculative search, but the addition of assumption dependence, nested federations, and an adaptive controller creates new composition and scheduling problems. We define $\mathsf{AMLD}^{+0}_{5\times\mathrm{FORGE6}}$ with settlement roles $\{P, R, D, I, C\}$, each realized by a FORGE-6 federation, and Player-0 $B^\star$. We distinguish semantic truth, checker acceptance, scoped dependence, and resource-bounded nonsettlement; separate a frozen BANK baseline from a provisional certified ledger; and formalize service authority, typed communication, and cost accounting. The proved results include scoped deterministic report arbitration, nonvacuous guard atlases, multi-federation noncontamination, context-safe transfer, finite guard-basis and guard-criticality theorems, passive emulation, two-level round-robin bounds under committed replay, conditional certificate-sharing gains, game-aware quotient preservation, and generator-independent promotion. An ATLAS-style repair pass corrected guard consistency, outcome arbitration, service-macrostep horizons, latency units, and report revocation. Counterexamples rule out automatic speedup, unsafe state quotienting, and universal safety claims from finite regression tests. Applications to eight security and engineering domains and a preregistered experimental program identify what remains open.

Illustration note. All illustrations are monochrome vector diagrams included directly in this editable LaTeX source. They illustrate formal architecture and conditional implications, not experimental execution.

Contents

Companion statement i

Abstract ii

1 Purpose, lineage, and claims 1
- 1.1 What changes and what does not . . . 1
- 1.2 Truth, certificates, and three possible sources of confusion . . . 1
- 1.3 Contributions and audit posture . . . 1

2 The thirty-one-component architecture 2
- 2.1 Five settlement roles and six outcome categories . . . 2
- 2.2 FORGE-6 as an exact role-local template . . . 2
  - 2.2.1 Role constraints without suppressing creative search . . . 2
- 2.3 The outer board-player $B^\star$ . . . 4
- 2.4 Typed semantic bus and authority . . . 4

3 Formal states, receipts, and admissible transitions 5
- 3.1 Frozen instance and working evidence . . . 5
- 3.2 Frozen scoped arbitration and revocable reports . . . 5
- 3.3 What does a certificate establish? . . . 6
- 3.4 The governing noncontamination theorem . . . 7

4 Scoped evidence and the Dependence federation 9
- 4.1 Why dependence needs its own certificate . . . 9
- 4.2 Minimal security assumptions as an antichain . . . 10
- 4.3 Preventing context laundering . . . 11

5 Player-0 control, emulation, and fairness 13
- 5.1 Passive rewiring . . . 13
- 5.2 A two-level fairness theorem . . . 13
- 5.3 Buses, activation, and charged time . . . 15

6 Cooperation, certification graphs, and cost 16
- 6.1 How five FORGE-6 federations cooperate . . . 16
- 6.2 Exactly when sharing saves measured verification work . . . 17
- 6.3 Fault containment across federations . . . 18

7 Controller-aware quotients and search **19**
7.1 Why a history quotient can break Player-0 19

8 Security-domain deductions and limitations **21**
8.1 The eight application families revisited 21
8.2 A scoped infrastructure theorem 21
8.3 Contradiction, quarantine, and false security 22
8.4 Finite Ocean suites and universal claims 22

9 Preregistered experiments and open problems **23**
9.1 No execution is asserted 23
9.2 Exact questions worth testing 23
9.3 Research boundary and conclusion 24

A Theorem audit and status register **25**

B Source and notation notes **27**

References **28**

1. Purpose, lineage, and claims

1.1 What changes and what does not

The underlying problem remains the same: a powerful search process may propose objects, but acceptance into trusted knowledge is a separate act. This sequel makes that principle robust under five simultaneous settlement disciplines, internal FORGE-6 cooperation, and external Player-0 control. We adopt the established FORGE-6 interpretation of an internal B_0 coordinator and five different mathematical searchers [4], and the established AMLD+0 convention $P_0 = B^\star$ with role-local authority [5]. The main claims are formal, conditional statements about this defined protocol. They are not discoveries of weaknesses in real cryptographic schemes or a proof of global AI safety.

1.2 Truth, certificates, and three possible sources of confusion

Three distinctions govern everything that follows.

1. A verifier accepting a byte string is not by itself a proof that the verifier, its model, or its axioms are sound. Semantic conclusions below explicitly assume soundness for the relevant certificate type and scope.
2. D is an active settlement role; UNKNOWN is a report of nonsettlement under the prescribed stopping conditions, not a search agent or a certificate that the target is independent.
3. An assertion true under an augmented context is not thereby true in a weaker, unscoped or differently labeled context.

1.3 Contributions and audit posture

The paper supplies a complete process decomposition, new guarded statements and proofs, cost accounting, and a test design. Where a conclusion needs fairness, faithful replay, decidable model checking, or a perfect lifting obligation, those hypotheses are stated rather than silently presumed. No theorem herein asserts unconditional settlement, polynomial cryptanalysis, alignment, or practical security of a deployed implementation.

2. The thirty-one-component architecture

2.1 Five settlement roles and six outcome categories

Definition 2.1 (Role and outcome alphabets)**.** Let

$$\mathcal{R} = \{P, R, D, I, C\}, \qquad \mathcal{O} = \{\mathsf{PROVED}, \mathsf{REFUTED}, \mathsf{DEPENDENCE}, \mathsf{INDEPENDENCE}, \mathsf{CONTRADICTION}, \mathsf{UNKNOWN}\}.$$

The five members of $\mathcal{R}$ are roles that generate candidate evidence. The sixth member of $\mathcal{O}$ is a bounded no-certificate outcome. The settlement report must identify the accepted certificate and its scope whenever it reports one of the first five categories.

P seeks a proof of T; R seeks a refutation of T; D seeks certified assumption-sensitive behavior; I seeks metatheoretically certified independence in a fixed formal setting; and C seeks a certified inconsistency of the declared assumptions. Roles may produce overlapping diagnostic evidence, but acceptance is always typed and priority rules are declared before a run. In particular, acceptance of a D certificate is not an unconditional proof of T or $\neg T$.

2.2 FORGE-6 as an exact role-local template

Definition 2.2 (A role-local FORGE-6 instance)**.** For $r \in \mathcal{R}$, set

$$\mathsf{F}^r = (B_0^r, B_1^r, B_2^r, B_3^r, B_4^r, B_5^r, \mathcal{M}^r, \sigma^r, \kappa^r, \mathcal{L}^r).$$

Here B_0^r is the internal coordinator, B_1^r backward ATP, B_2^r forward ATP, B_3^r adversarial ATP, B_4^r proof-geometry/zipper ATP, and B_5^r counterfactual ATP. $\mathcal{M}^r$ is a typed message bus, σ^r an internal scheduler, κ^r a cost ledger, and $\mathcal{L}^r$ the role-local record of candidates and certified receipts. All five mathematical ATPs may use different search tactics but none can alter the target, context, verifier, or admission rule.

2.2.1 Role constraints without suppressing creative search

The outer role dictates the *type of certified result*, not a single internal search style. For example, B_3^P may search for weaknesses in a proposed proof and B_5^R may hypothesize a counterexample, but only the correctly typed receipt makes an outer P or R settlement. Similarly, B_1^D searches backward for a minimal dependency, B_2^D mines useful conditional lemmas, B_3^D challenges spurious assumptions, B_4^D navigates assumption lattices, and B_5^D explores conditional alternatives. This is a design assignment, not a report that these processes have run.

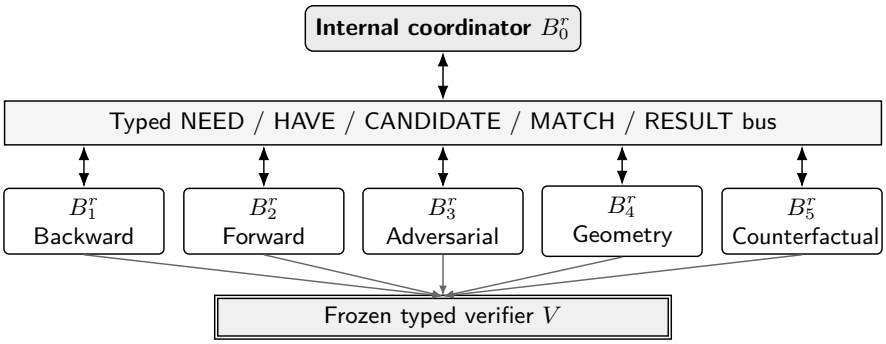


Figure 2.1: The six distinct components of one FORGE-6 federation. The full AMLD+0 sequel instantiates this template five times, one per settlement role.

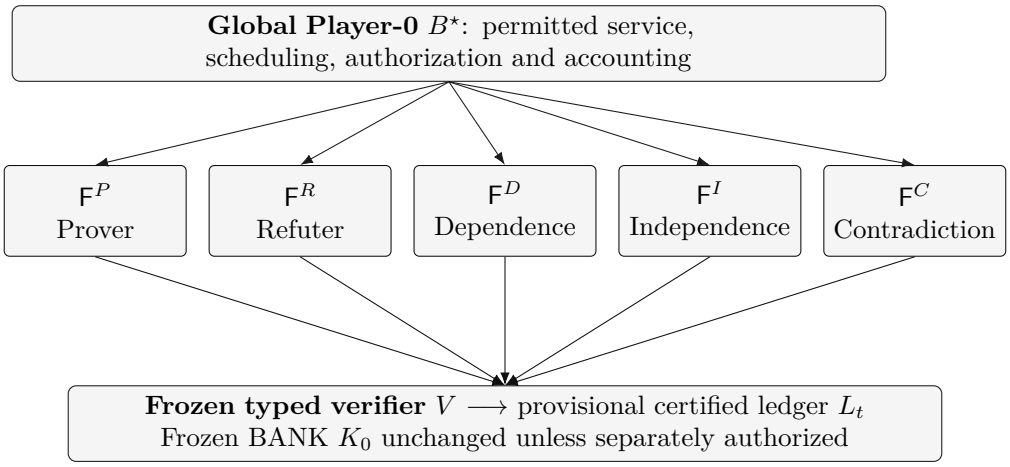


Figure 2.2: The two-level AMLD+0 architecture. Global Player-0 $B^\star$ dispatches five role-local FORGE-6 federations; the fixed verifier controls admission to the provisional certified ledger.

Proposition 2.3 (Typed cardinality)**.** *If every* F^r *has one internal non-proving coordinator and five distinct mathematical ATP components, and if* $B^\star$ *is distinct from all of them, then the architecture has* $5 \cdot 5 = 25$ *mathematical ATPs, five internal coordinators and one global Player-0, hence* 31 *components.*

Proof. Five disjoint role tags each index five ATPs and one coordinator; $B^\star$ is a separately tagged controller. Sum the disjoint sets. □

Corollary 2.4 (Not thirty-one votes)**.** *No count of components confers proof authority: certificates are accepted by* V*, not by majority or agreement among any of the* 31 *components.*

2.3 The outer board-player $B^\star$

The symbol $B^\star$ names the *active global Player-0*, not BANK, not a stored board state, and not any B_0^r. At its legal turn $B^\star$ may choose $\mathsf{Serve}(r)$ for enabled $r \in \mathcal{R}$ or a permitted delegation ε. The chosen federation and its internal B_0^r then choose legal internal work according to their policies. $B^\star$ may reason about which permitted task to pursue, but does not choose an ATP's internal proof step or assert a role certificate on its own authority.

An extended protocol may allow board/federation activation only upon an explicitly authorized $B^\star$ turn and subject to credits, frozen rules, and fully charged resources. Activation is *not* automatic, and the 31-component core does not require extra spawned instances.

2.4 Typed semantic bus and authority

Legal packet types are NEED, HAVE, CANDIDATE, MATCH, STATUS, RESULT, and WITHDRAW (and versioned extensions such as FAIL). Every packet carries a role tag, originating ATP, target, context, dependency digest, trust status, destination and resource charge. Internal buses are initially OFF; $B^\star$ can explicitly authorize a bus in the declared mode and can disable it. Cross-federation packets pass a typed gateway and cannot self-promote.

3. Formal states, receipts, and admissible transitions

3.1 Frozen instance and working evidence

Definition 3.1 (Frozen instance)**.** A run is parameterized by

$$\mathcal{I} = (T, \Gamma_0, E_0, V, \mathfrak{C}, K_0, \mathcal{A}_0, \mathbb{B}, \sigma_{\text{base}}, \rho, \Pi_0),$$

where T is the target; Γ_0 is the admitted theory; E_0 is the environment; V the fixed verifier (including certificate formats); $\mathfrak{C}$ the allowed context, assumption and label family; K_0 the frozen certified BANK; $\mathcal{A}_0$ the frozen transition authority; $\mathbb{B}$ the global charged resource budget; σ_{base} the baseline scheduler; ρ the seed or other replay-determining data; and Π_0 the immutable, scoped settlement-reporting and revocation policy defined below. No component may silently revise these.

Definition 3.2 (Accepted tuple and provisional ledger)**.** A checked object is a tuple

$$z = (\text{type}, c, \gamma, e, \lambda, \pi, V\text{-version}, \text{deps}, \text{origin}, \text{cost}).$$

The tuple's γ identifies its exact assumption context, e its environment and λ its authority or secrecy label. An object is *certified* when the fixed typed verifier accepts it and all referenced dependencies are certified in compatible scopes. Let L_t be an append-only *provisional certified ledger*; its records do not overwrite K_0. Unchecked material belongs to U_t. The effective read-only verified view, where access is authorized, is $K_0 \cup L_t$, not an unlogged mutation of K_0.

A deployment profile can authorize separately gated write-back to a versioned persistent BANK, but is not assumed here. This distinction allows a frozen experimental baseline and learning during a run without silently changing its admitted premises.

Definition 3.3 (Rule-complete global state)**.** A state consists of all thirty-one local control/search states, the frozen instance, L_t, U_t, bus-enable bits, role permissions, pending checker calls, provenance DAG, accepted typed outcomes, clocks, resource accounts and scheduler/seed state sufficient to determine legal successors. A *legal move* is a transition allowed by $\mathcal{A}_0$ that preserves the immutable instance and obeys its verification and budget checks.

3.2 Frozen scoped arbitration and revocable reports

Definition 3.4 (Frozen report arbiter and scope)**.** A *scope* γ fixes theory, target, model class, environment, versioned verifier, authority label and relevant dependency identifiers. The frozen

policy Π_0 includes (i) exact-scope matching and verified transfer rules; (ii) certificate eligibility, including a separate satisfiability receipt before display of a *nonvacuous safety* claim; (iii) a deterministic precedence rule; (iv) terminal and stopping conditions; and (v) quarantine, supersession and revocation. Let $\mathcal{E}_t(\gamma)$ be the time-stamped vector of admitted P, R, D, I, C receipts eligible for scope γ, *not* a single truth value. All receipts remain in the append-only evidence ledger. For definiteness, this paper freezes the primary-display order

$$C \succ R \succ P \succ D \succ I,$$

with accepted-event order followed by canonical receipt digest as its deterministic tie-breaker. This order is an administrative display convention, not a logical implication. The public record additionally contains the full eligible vector and each receipt's exact scope.

Definition 3.5 (Reporting, pending, unknown and quarantine)**.** Define $\mathrm{Report}_{\Pi_0}(\mathcal{E}_t, \gamma, t)$ by: (a) a verified C receipt at γ or in a certified weaker context whose theory extends into γ marks the relevant scope QUARANTINED, and revokes every previous *safety display* in that affected scope; (b) otherwise eligible receipts are shown as a full vector with a primary display chosen by the frozen order; (c) if none is eligible and either the stop rule has not fired or mandatory checker calls remain pending, return PENDING; (d) if none is eligible, the stop rule has fired, and no mandatory checker call remains pending, return UNKNOWN. Quarantine is a reporting status, not a seventh settlement role. A revoked report remains in the historical log tagged REVOKED, never silently erased. Scope propagation requires a checked extension relation with the same logic and environmental meaning; unrelated scopes remain untouched. The protocol creates versioned replacement reports or revocation notices, but cannot retract copies already exported to ungoverned external systems.

Theorem 3.6 (Deterministic scoped report and revocation)**.** *If the accepted ledger, immutable scope-compatibility data, event sequence, stop status and* Π_0 *are identical, then the report vector, primary display, quarantine and revocation flags are identical. An accepted C receipt cannot, by this rule, revoke reports in an unrelated scope, but it invalidates all affected prior safety displays.*

Proof. Every predicate, precedence comparison and digest tie-break is a fixed function of those inputs. Quarantine is computed from checked scope extension; affected historical safety entries receive a revocation tag. Unrelated entries fail that extension predicate. □

Corollary 3.7 (Unknown is neither a role nor a timeout theorem)**.** *UNKNOWN is reported only for the specified scope after its stop rule fires, with no eligible certificate; it implies neither independence nor falsity. Pending checks instead give PENDING.*

Remark 3.8 (Inconsistency and certificate validity). A C receipt does not erase proofs from the immutable evidence ledger. It may render them unsafe for *nonvacuous safety reporting*. A semantic soundness failure or changed checker is not repaired merely by report precedence; such incidents require separate trust reassessment.

3.3 What does a certificate establish?

The verifier is a family of typed checks $V_\tau(\pi, c \mid \gamma, e)$. For P, a receipt proves the target in the specified context. Displaying it as nonvacuous security additionally requires the declared

model-satisfiability obligation. For R, a proof of its negation or an admitted semantically sufficient counterexample is checked. For I, the proof or pair of models must establish the declared independence claim in the fixed metatheory; failure of P and R searches is not evidence of I. For C, a receipt certifies inconsistency of the specific context. For D, a receipt certifies stated context sensitivity, defined precisely in Chapter 5. The checker must not confuse these types. Raw V checking uses the full immutable input; ledger admission additionally needs currently certified dependencies, an unexpired scope key and the frozen Π_0 eligibility rules.

Definition 3.9 (Soundness hypothesis)**.** A verifier is *sound on an admitted scope family* if every accepted receipt supports exactly its stated semantic or metatheoretic conclusion in the declared model and scope. We do not assume that acceptance by an arbitrary implementation implies this hypothesis.

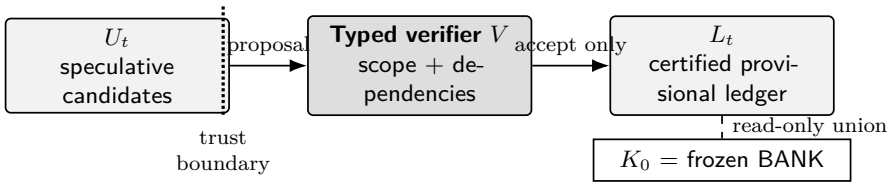


Figure 3.1: No packet, role decision, vote or Player-0 instruction crosses the trust boundary as a certified assertion without verification. This model deliberately keeps the original BANK K_0 frozen.

3.4 The governing noncontamination theorem

Theorem 3.10 (Federated noncontamination)**.** *Suppose (i) L_0 is certified; (ii) the only ledger insertion operation appends a V-accepted, scope-valid, support-closed tuple; and (iii) all other moves, including all* 31 *components' actions and packet transmissions, leave L unchanged. Then every L_t contains only certified tuples, and no member of U_t enters L_t merely by being transmitted, repeated, scored highly or proposed by $B^\star$.*

Proof. Induct on legal global transitions. The base case is assumption (i). The only transition that enlarges the ledger satisfies (ii). Every other transition preserves the inductive invariant by (iii). The provenance of the proposal is irrelevant to the acceptance check. □

Corollary 3.11 (Twenty-five-ATP immunity to voting)**.** *Even unanimous agreement of the twenty-five mathematical ATPs cannot promote an unsupported candidate in this protocol.*

Corollary 3.12 (Conditional semantic soundness)**.** *If, additionally, V is sound on all accepted scopes and the meaning of each accepted tuple is interpreted in its declared scope, every member of L_t has its certified typed meaning. This does* not *imply that the admitted environment models the deployed world.*

Theorem 3.13 (Verifier-neutral controller replacement)**.** *Suppose two runs differ only in $B^\star$'s scheduling policy, a FORGE-6 internal search policy, or bus routing, while V, the certificate language, frozen contexts and ledger insertion rule are the same. For every* identical candidate tuple *presented to V, its raw verification result is identical in both runs if the complete checker input, including referenced content hashes, is identical. Admission to L_t is a separate state-dependent decision and may differ because dependency availability, budget and quarantine differ. The runs may also differ in candidates and resource use.*

Proof. The typed verifier is a fixed function of the complete frozen verification input, not its proposing policy. Ledger admission may depend on an additional changing dependency-availability predicate. □

Corollary 3.14 (Policy improvement is not truth improvement)**.** *A changed schedule can improve certificate discovery rates but cannot make an invalid certificate valid under the same frozen checker.*

4. Scoped evidence and the Dependence federation

4.1 Why dependence needs its own certificate

A claim that changes with an assumption is not the same as a claim independent of an axiomatic theory. The role D investigates certified relationships between *different* admissible contexts; I investigates non-derivability of a fixed statement and its negation from a *fixed* theory under the chosen metatheory. Neither role may derive its conclusion from elapsed time or failed search alone.

Definition 4.1 (Admissible nonvacuous guard family)**.** Fix a base theory Γ_0, a finite optional guard set $G = \{g_1, \ldots, g_n\}$, an unchanged object-language interpretation, and a common declared model class. For $A \subseteq G$ define $\Gamma_A = \Gamma_0 \cup A$ and

$$\mathcal{C} = \{A \subseteq G : \mathsf{Sat}(\Gamma_A)\}, \quad \mathcal{S}_T = \{A \in \mathcal{C} : \Gamma_A \models T\}.$$

Only $A \in \mathcal{C}$ are *admissible security guard contexts*. The semantic predicate $f_T(A)$ is defined on $\mathcal{C}$ by $f_T(A) = 1$ iff $\Gamma_A \models T$; $f_T(A) = 0$ means that a model of $\Gamma_A \wedge \neg T$ exists. For $A \notin \mathcal{C}$ the security predicate is *inadmissible*, not true by explosion. Certification of $A \in \mathcal{C}$ requires a sound, checked model witness or a separately justified, checkable consistency certificate. For arbitrary infinite theories such a finite checker may not exist; no universal decision procedure is presumed.

Definition 4.2 (Strong guarded D-receipt)**.** A strong D-receipt for T at guards $A_- \subsetneq A_+ \subseteq G$ consists of: (i) a checker-accepted proof of T from Γ_{A_+}; (ii) a checker-accepted model of Γ_{A_+} (to exclude vacuous proof from inconsistency); (iii) a checker-accepted countermodel to T satisfying Γ_{A_-}; (iv) verified identity of target meaning across the two contexts; and (v) the declared environment, scope and guard provenance. It establishes $f_T(A_+) = 1$ and $f_T(A_-) = 0$ in a nonvacuous stronger context. It does not establish $\Gamma_0 \models T$ or $\Gamma_0 \models \neg T$ unless separately proved.

This is deliberately a *strong* operational certificate schema. A weaker *conditional sufficiency record* consists of a checked proof of T at an admissible, satisfiable guard A but has no certified negative comparison; it is useful D-role evidence but is *not* yet a DEPENDENCE settlement. A *deletion-essentiality record* adds the checked countermodel at $A \setminus \{g\}$ and is a strong D receipt for that guard. Consequently the strong D-to-semantic-I bridge below need not apply to every preliminary or weak D record. No general implication between other meanings of dependence is presumed.

Theorem 4.3 (Sound guarded dependence)**.** *If a strong D-receipt is accepted by a sound typed verifier and its contexts satisfy the stated guard semantics, then entailment of T differs between*

its certified contexts. Its promotion as a context-free P or R settlement is unjustified unless a separate matching receipt is accepted.

Proof. The proof component gives $f_T(A_+) = 1$. The countermodel component witnesses $f_T(A_-) = 0$. Neither component proves the target in any unnamed context, so a change of certificate type or scope requires another inference and check. □

Corollary 4.4 (Strong dependence is not independence)**.** *A certified D-receipt does not, without a checked conversion into the declared I-certificate language, constitute an accepted I settlement, even when its models are informative about Γ_0.*

Theorem 4.5 (Dual-model bridge from strong dependence)**.** *Under the same-model-class interpretation of the guard contexts, a strong D-receipt with its consistency model M_+ and deletion-side countermodel M_- gives two models of Γ_0, one satisfying T and the other satisfying $\neg T$. Therefore Γ_0 semantically entails neither T nor $\neg T$. If the frozen I verifier accepts this pair as a sufficient independence certificate, it can also establish an I settlement after a separate typed check.*

Proof. Both Γ_{A_+} and Γ_{A_-} extend Γ_0. The model M_+ satisfies T because the sound proof establishes $\Gamma_{A_+} \models T$. The model M_- satisfies $\neg T$ by its verified counterexample. Both models satisfy Γ_0, so neither proposed entailment from Γ_0 holds. The final sentence follows only under the stated I-receipt rule and check. □

Remark 4.6 (Overlap of evidence is not collapse of roles)*.* This bridge is a new consequence of the nonvacuous strong guard semantics chosen for D here. It illustrates how different settlement roles can share evidence, not that D and I have identical objectives or that every dependence framework yields independence.

4.2 Minimal security assumptions as an antichain

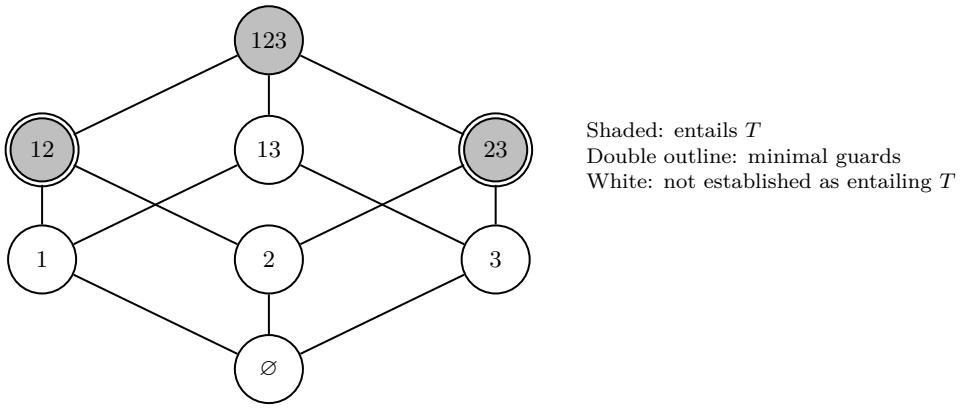


Figure 4.1: Illustrative three-guard lattice, written $12 = \{g_1, g_2\}$ and similarly for other nodes. All displayed guard contexts are assumed satisfiable. The assumed minimal nonvacuous sufficient sets are $\{g_1, g_2\}$ and $\{g_2, g_3\}$. This is a formal example, not empirical evidence.

Theorem 4.7 (Finite nonvacuous minimal-guard basis)**.** *For finite G, the satisfiable family $\mathcal{C}$ is downward closed in 2^G, and the nonvacuous success family $\mathcal{S}_T$ is upward closed* relative to $\mathcal{C}$*. Its unique antichain $\mathcal{M}_T$ of inclusion-minimal elements satisfies*

$$\mathcal{S}_T = \{A \in \mathcal{C} : \exists M \in \mathcal{M}_T \ (M \subseteq A)\}.$$

No inconsistent A enters this atlas, even when classical explosion makes $\Gamma_A \models T$ true.

Proof. Deleting optional assumptions preserves satisfiability, so $\mathcal{C}$ is downward closed. If $A \in \mathcal{S}_T$, $A \subseteq B$ and $B \in \mathcal{C}$, then any model of Γ_B models Γ_A, hence $\Gamma_B \models T$. For every successful A, finite descent inside $\mathcal{C}$ gives a minimal successful subset; the preceding relative monotonicity proves the reverse inclusion. Minimal elements are unique and incomparable. Exclusion of $A \notin \mathcal{C}$ is by definition. □

Corollary 4.8 (Certified nonvacuous sufficient-condition atlas)**.** *An entry M can be promoted only with (i) a checked entailment receipt for T at Γ_M, (ii) a checked satisfiability/consistency receipt for Γ_M, and (iii) checked negative entailment witnesses for all proper relevant subcontexts, for example immediate deletions. A* complete *atlas additionally requires certified exhaustive coverage of the finite guard lattice. Without negative and coverage checks, a collection of successful entries is only a candidate atlas.*

Example 4.9 (Explosion rejected by the atlas)**.** Let $\Gamma_0 = \{p\}$, $G = \{\neg p\}$ and $T = q$ in classical propositional logic. The apparent "success" of $\{\neg p\}$ under unrestricted entailment is vacuous: $\Gamma_0 \cup \{\neg p\}$ has no model. Therefore $\{\neg p\} \notin \mathcal{C}$ and it never appears in $\mathcal{M}_T$.

Theorem 4.10 (Deletion-witness minimality)**.** *Let $M \subseteq G$ have a certified proof of T under Γ_M and a checked satisfying model of Γ_M. If, for every $g \in M$, there is a certified model of $\Gamma_{M\setminus\{g\}} \wedge \neg T$, Then M lies in $\mathcal{M}_T$, the nonvacuous inclusion-minimal sufficient guard family. If $M \neq \varnothing$, then for each $g \in M$ a strong D-receipt is furnished by the proof for M and the corresponding deletion countermodel.*

Proof. Each immediate deletion fails to entail T. Any proper subset $A \subsetneq M$ lies inside at least one $M \setminus \{g\}$; its set of models includes the verified countermodel for that deletion, which therefore refutes entailment at A. Thus no proper subset of M is sufficient. The satisfying model of Γ_M rules out vacuous success, and the receipt pairs satisfy the strong D schema. □

Corollary 4.11 (Criticality is explicit)**.** *The D federation can name a guard g as critical to a minimal certificate only with a checked deletion witness, not merely because B_5^D imagined the guard absent or B_1^D requested it.*

Example 4.12 (Two independent protections)**.** Let a toy authorization event be safe exactly when guards g_1 ("strong authentication") or g_2 ("physical isolation") is present in a stipulated Boolean model. Then $f_T(A) = 1$ iff $g_1 \in A$ or $g_2 \in A$, and $\mathsf{Min}_T = \{\{g_1\}, \{g_2\}\}$. The empty guard set has a direct countermodel; each singleton has a trivial Boolean proof. This is a model-relative example of an antichain, not a real system audit.

4.3 Preventing context laundering

Definition 4.13 (Scope-safe use)**.** A certified tuple established under γ may be used in δ only if a checked transfer operation licenses the use $\gamma \rightsquigarrow \delta$, including required theory, label,

environment, dependency and certificate-lifting obligations. A HAVE packet with a proof from γ is not such a transfer by itself.

Theorem 4.14 (No unlicensed cross-context promotion)**.** *Suppose every local inference either (a) uses premises with matching scopes or (b) includes a verified scope-transfer receipt, and every ledger insertion checks this property recursively through its support DAG. Then no ledger proof of T at scope δ can depend on a foreign-scope premise without a verified chain of transfers from that premise to δ.*

Proof. Induct over the finite acyclic dependency graph for the inserted certificate. Leaves are appropriately scoped axioms or transferred premises by the insertion rule. For an internal inference, the rule ensures that each parent is locally scoped or arrives by a verified transfer. Concatenating parent chains gives the required chain for every dependency. □

Corollary 4.15 (Provenance-preserving federated reuse)**.** *Shared ledgers may reduce repeated proof work without equating the authorization labels, hypotheses, or environmental assumptions of different security domains.*

5. Player-0 control, emulation, and fairness

5.1 Passive rewiring

Definition 5.1 (Faithful baseline delegation)**.** A baseline run is *faithfully embeddable* if an explicit embedding e_r maps each legacy role-local state into one designated B_j^r state, with a committed, input-preserving step interpreter that simulates each recorded transition and returns the same checked receipt and context. The baseline scheduler choices can be reproduced by $B^\star$'s legal ε-delegation or $\mathsf{Serve}(r)$ actions, the chosen F^r executes each recorded local move under a committed replay policy, other components may remain inactive without changing those moves, and verification and resource allowances suffice for the replay including extra administration. The interpreter and its cost are data of the embedding, not assumed to exist for every arbitrary legacy ATP.

Theorem 5.2 (Passive embedding with charged overhead)**.** *If a legacy four- or five-role certified trace is faithfully embeddable, there exists a run of the new architecture whose projection onto the legacy role states and certified outcomes is that trace. If legacy cost is C, the embedded run needs budget at least $C + H$, where $H \geq 0$ is its measured added control, routing and bookkeeping cost. No equal-budget simulation is asserted when $H > 0$.*

Proof. At each baseline service event let $B^\star$ delegate or select its recorded role and let the internal federation perform the recorded committed move. All other components can idle legally. The specified step interpreter witnesses each simulated transition. Induction on events gives the same projected state and checker decisions. The new control and accounting actions add the measured H. □

Corollary 5.3 (Reachability inclusion, not unconditional efficiency)**.** *At adequately increased budget the new architecture can preserve old certified reachability. This alone proves neither better discovery rates nor strict cost dominance at the same budget.*

5.2 A two-level fairness theorem

A precise bound requires an actual scheduling policy, not a slogan about fair opportunity. Define a global *service tick* as an authorized dispatch to one outer federation that results in exactly one counted local ATP transition. Local verifier time is separately charged; if verification is asynchronous, its delay is tracked.

Definition 5.4 (Two-level cyclic service)**.** The global controller visits P, R, D, I, C in a fixed round-robin cycle. On its kth service, each F^r dispatches exactly one of its five mathematical ATPs in a fixed cycle. Thus every ordered pair $(r, j) \in \mathcal{R} \times \{1, \ldots, 5\}$ receives exactly one

	ATP 1	ATP 2	ATP 3	ATP 4	ATP 5
F^P	B_1^P	B_2^P	B_3^P	B_4^P	B_5^P
F^R	B_1^R	B_2^R	B_3^R	B_4^R	B_5^R
F^D	B_1^D	B_2^D	B_3^D	B_4^D	B_5^D
F^I	B_1^I	B_2^I	B_3^I	B_4^I	B_5^I
F^C	B_1^C	B_2^C	B_3^C	B_4^C	B_5^C

Under executed two-level cyclic service, a designated B_j^r receives one local step per 25 global ticks.
Committed baseline replay and adequate checker/budget allowances remain necessary for a settlement bound.

Figure 5.1: Five role-local federations × five mathematical ATPs form 25 distinct service channels. The diagram illustrates the fairness accounting, not simultaneous 25-step execution.

executed transition in every block of 25 global service ticks, provided the run does not terminate early and every scheduled move is enabled.

Theorem 5.5 (Twenty-five-channel committed-replay bound)**.** *Fix a pair (r, j) with a checker-accepted target-typed certificate after $m \geq 1$ local transitions of a frozen baseline policy. Assume: (i) two-level cyclic service executes one local transition in each scheduled slot until settlement; (ii) the designated ATP implements the successful committed baseline policy; (iii) other work leaves its state projection, scope and verifier unchanged; and (iv) the budget covers $25m$ dispatched transitions and all mandatory checks. Then the certificate is* submitted *by dispatch count $N_{\text{dispatch}} \leq 25m$, unless a qualifying certified outcome terminates the run earlier. If a fully serviced dispatch, including all mandatory intermediate administration but not the final check, consumes elapsed time at most $d_{\max}$, idle waiting is included in that bound, and the final checker latency is at most τ_V, then*

$$t_{\text{settlement}} - t_{\text{start}} \leq 25m\, d_{\max} + \tau_V .$$

Absent such duration and verifier bounds, the dispatch guarantee implies no finite wall-clock guarantee.

Proof. Every block of 25 executed dispatches contains one transition for (r, j). With no earlier accepted termination, m blocks supply m designated moves, which by committed replay generate the proof candidate. This establishes the discrete dispatch bound. For the elapsed-time claim add the worst-case time for at most $25m$ charged dispatch intervals and the separately bounded final checker call. □

Corollary 5.6 (Role-local bound)**.** *If one only round-robins the five inner ATPs and the outer controller services the role at every tick, the corresponding dispatch bound is $5m$, under the same replay assumptions.*

Corollary 5.7 (No bound from fair offers alone)**.** *Replacing "executes a transition" by "offers an opportunity" in Theorem 5.5 invalidates the conclusion: an ATP may idle or choose another admissible move forever.*

Example 5.8 (Necessity of commitment)**.** Let an internal prover have two legal actions, finish (producing a proof of T in one step) and idle (returning to itself). A perfectly fair controller may serve it infinitely often; a local policy always choosing idle never settles.

5.3 Buses, activation, and charged time

Service ticks are not total work or necessarily wall-clock seconds. A resource accounting convention is

$$C_{\text{total}} = C_{\text{search}} + C_{\text{verify}} + C_{\text{route}} + C_{\text{schedule}} + C_{\text{activation}} + C_{\text{inherit}} + C_{\text{admin}}.$$

Every positive event appears once in its cost category; no free oracle, invisible bus or free spawn is allowed. If the bus is OFF, there is no cross-role communication through it; if enabled, each message is typed, logged and charged. $B^{\star}$ can activate a permitted child only on an authorized turn and with adequate credit. Neither an extra child nor a new lemma may be written silently into frozen K_0.

6. Cooperation, certification graphs, and cost

6.1 How five FORGE-6 federations cooperate

An internal B_1^r may emit $\mathsf{NEED}(\ell)$, an internal B_2^s may emit $\mathsf{HAVE}(\ell)$, and B_4 searchers can identify theorem-graph proximity. B_3 can challenge the bridge; B_5 can explore a conditional route. A common, typed certificate for ℓ can then be reused across legal boundaries only after scope and dependency checks. The important object is a checked proof-DAG with labeled edges, not an unrestricted union of claims.

Proposition 6.1 (Composite deduction without new axioms)**.** *Let two certified local collections at a* common scope *be $K_1 = \{a\}$ and $K_2 = \{a \to b\}$, with ordinary propositional consequence. Then $b \in \mathrm{Cn}(K_1 \cup K_2)$ but $b \notin \mathrm{Cn}(K_1)$ and $b \notin \mathrm{Cn}(K_2)$.*

Proof. Modus ponens yields b jointly. A model with a true and b false satisfies K_1; a model with both false satisfies K_2. Thus neither collection separately entails b. □

Remark 6.2. This is an example of *distributed knowledge synergy*, not a new theorem relative to the full frozen premises, which may already entail b. It also differs from a multiarticle transcendent theorem, which requires non-derivability from each *full individual closure*. The distinction must not be blurred.

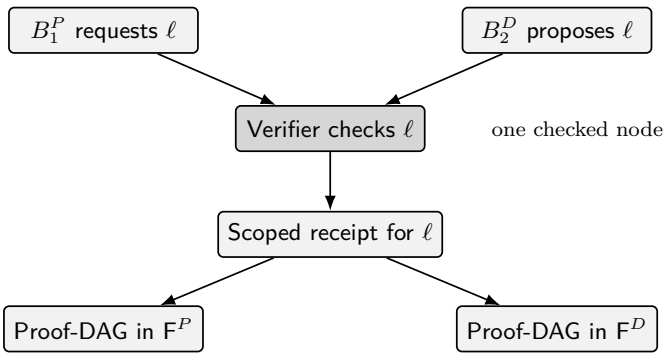


Figure 6.1: Typed NEED/HAVE cooperation can reuse an identical, scope-compatible certified proof node in multiple investigations. Cost savings arise only when avoided verification work exceeds routing and bookkeeping overhead.

6.2 Exactly when sharing saves measured verification work

Definition 6.3 (Identical certified proof nodes)**.** A proof node x includes its normalized statement, inference rule, full context, verifier version, dependencies and label. A canonical cache key includes a byte-exact normalized statement, proof/certificate bytes, transitive dependency digests, inference rules, theory and environment digests, verifier version, authority label and scope. A digest lookup is an index, not a license to trust hash collisions: identity is checked by collision-resistant assumptions plus byte-level equality or an equivalent authenticated representation. Changing any field invalidates the corresponding key. Cached results remain subject to scope checks, dependency replay, revocation and cache-read verification charges; these costs belong in H below. Nodes with the same surface text but different scope or authority are *not* identical and cannot silently share certified status.

Theorem 6.4 (Conditional DAG-reuse saving)**.** *Suppose k standalone investigations have certified proof-node DAGs $D_1, \dots, D_k$ and identical-node verification costs $v(x) \geq 0$. If checking every node afresh costs*

$$C_{\text{sep}} = \sum_{i=1}^{k} \sum_{x \in D_i} v(x),$$

and shared proof-DAG checking costs

$$C_{\text{shared}} = \sum_{x \in \bigcup_i D_i} v(x) + H,$$

where H is all incremental routing, deduplication, scheduling, key comparison, cache replay, invalidation, lookup and certification overhead, then

$$C_{\text{sep}} - C_{\text{shared}} = \sum_{x \in \bigcup_i D_i} (m_x - 1)v(x) - H,$$

where m_x is the number of standalone DAGs containing x. Consequently shared checking is cheaper if and only if *the avoidable repeated-check cost exceeds H.*

Proof. Count the verification charge of each distinct node m_x times in C_{sep} and once in C_{shared}; subtract H. □

Corollary 6.5 (No assumed federation speedup)**.** *Twenty-five ATPs can increase overhead faster than they save checking. Parallelism, certificate sharing and better guidance require separate measured evidence to establish total-cost improvements.*

Theorem 6.6 (No unconditional cost dominance)**.** *If the global architecture incurs any strictly positive unavoidable startup cost $h > 0$ relative to a standalone ATP, then there are target families for which it cannot strictly beat the standalone ATP in* total charged work*, even while faithfully reproducing the standalone proof.*

Proof. Choose a target whose complete verified certificate is already available in the initial trusted context, so a standalone baseline may return it with no proof-search work (or the minimum mandatory reporting charge). Any extra unavoidable startup charge h of the new protocol cannot lower that minimum. Thus universal strict cost dominance cannot follow merely from adding the architecture. □

6.3 Fault containment across federations

Theorem 6.7 (Candidate fault isolation)**.** *Assume a faulty* F^r *can generate arbitrary proposals and packets but cannot modify* V*, its admission predicate, other roles' private states, or the certified ledger except through the common gate. Then that federation cannot, by proposal alone, add an unaccepted assertion to the ledger.*

Proof. Its actions are among the non-insertion moves of Theorem 3.10; its attempted insertions are gated. □

Remark 6.8. This is an *integrity isolation* result, not a liveness or availability guarantee. A faulty federation can still consume resources or starve useful work if scheduling and budgets allow it.

7. Controller-aware quotients and search

7.1 Why a history quotient can break Player-0

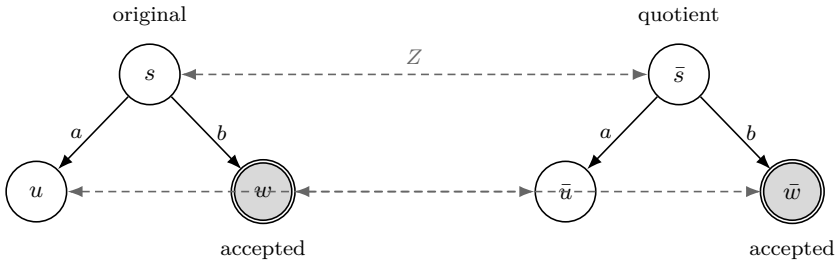


Figure 7.1: A schematic controller-respecting correspondence Z: matching authorized actions and successor behavior plus matching accepted-target labels are necessary for game-aware reuse. This is an illustration of the hypotheses, not proof of a particular quotient.

The original security manuscript requires future-sufficient compression. In a controller game, merely matching terminal labels is insufficient: a quotient must also preserve which service choices remain legal and the possible reactions after each choice.

Definition 7.1 (One-service macro-step and stalling)**.** A *stable boundary* is a rule-complete state at which $B^\star$ may choose the next charged service. Write

$$s \overset{a}{\Longrightarrow} s'$$

if one legal $B^\star$ action a charges exactly one service and, after its prescribed zero-service bus, checking, ledger, quarantine and arbitration microsteps, reaches the next stable boundary s'. Intermediate accepted target status is recorded in s'. A timeout, infinite administrative loop or checker divergence is represented by a dedicated nonaccepting losing sink $\bot$, which cannot be passed off as a successful boundary. Each enabled action has at least one macro-successor including $\bot$ if necessary. All nondeterministic macro outcomes are included. A run that continuously performs such macrosteps receives no uncharged external changes in between them. The "one move" bound below counts exactly these macrosteps, *not* checker microsteps or elapsed time.

Definition 7.2 (Controller-respecting bisimulation)**.** Let $\mathcal{G}$ and $\bar{\mathcal{G}}$ be turn-based, nondeterministic settlement transition systems, with target-accepting terminal sets W and $\bar{W}$. A relation Z between states is a *controller-respecting bisimulation* if, whenever $sZ\bar{s}$: (i) $s \in W$ iff $\bar{s} \in \bar{W}$; (ii) the enabled $B^\star$ service-action labels agree (up to a fixed verified renaming); and (iii) for each

corresponding action a, every *macro-successor* of s under a is related to a macro-successor of $\bar{s}$ under a, and conversely. The relation contains any distinguished timeout/divergence sink and matches nonaccepting sink labels. Raw checker/bus microsteps are included in the macrostep closure, not counted as service actions.

Theorem 7.3 (Finite-horizon game-aware quotient preservation)**.** *Let Z be a controller-respecting bisimulation between states $sZ\bar{s}$. For each finite horizon $n \geq 0$, $B^\star$ has a strategy forcing an accepted target state within n service moves against every admitted macro-successor choice from s if and only if the corresponding statement holds at $\bar{s}$.*

Proof. Define $W_0 = W$ and recursively

$$W_{n+1} = W_n \cup \{s : \exists a \in A(s)\ \forall s' \in \mathrm{Succ}(s,a),\ s' \in W_n\},$$

where $\mathrm{Succ}(s,a) = \{s' : s \overset{a}{\Rightarrow} s'\}$ contains all completed service-macro outcomes, including failure sinks, and is nonempty for enabled a; define $\bar{W}_n$ analogously. For $n = 0$ the claim is clause (i). Assume the claim for n. Corresponding enabled actions and both directions of successor matching in clause (iii), together with the induction hypothesis, make the existential-action/universal-macro-successor clause invariant under Z. Hence membership in W_{n+1} agrees, and induction proves the claim. □

Corollary 7.4 (Safe strategic state merging)**.** *A quotient whose classes satisfy this stronger relation preserves finite-horizon forced certified settlement. Mere equality of a search heuristic, local proof count, or current terminal status does not suffice.*

Example 7.5 (An unsafe tempting quotient)**.** Two states are both unsettled and have identical current certified BANK. From the first the controller can service P to obtain an accepted certificate in one move; from the second P is permanently disabled. Merging these states because their present ledger is equal loses the one-step winning distinction.

Remark 7.6 (Scope and cost must survive quotienting). If the target is a minimal-cost or probabilistic guarantee rather than finite-horizon reachability, one needs additionally cost-preserving transfers or appropriate kernel-equivalence and optimization hypotheses. Theorem 7.3 proves neither speedup nor preservation of probability distributions.

8. Security-domain deductions and limitations

8.1 The eight application families revisited

Domain	New D-federation question	New $B^\star$/FORGE-6 obligation
Cryptography	Under which exact structural assumptions is a quotient sound?	Controller must preserve certificate lifting and avoid assuming an attack from compression.
Vulnerability research	Which guard or policy clause is essential to a defect/safety certificate?	Route checked traces and proofs without mixing implementation models.
Critical infrastructure	Which segmentation cuts are necessary for bounded blast radius and service?	Schedule containment and availability checks with explicit nonregression.
Compartmentation	Which label, declassification or authority assumption supports noninterference?	Keep cross-domain certificate labels, provenance and policy context distinct.
Software correctness	Which environmental assumptions make an invariant inductive?	Coordinate invariants/counterexamples while retaining executable gate invariants.
Scientific discovery	Which observation family and model assumptions support sufficient quotients?	Rank hypotheses, but do not turn predictive fit into unconditional theorems.
Engineering optimization	Which constraints preserve feasible continuations and optimum values?	Price checker overhead and preserve downstream objective obligations.
AI safety	Under which verifier, deployment and scope assumptions is a generated claim promotable?	No role or controller may rewrite its own acceptance predicate.

8.2 A scoped infrastructure theorem

Theorem 8.1 (Typed cut-and-service nonregression, model relative)**.** *Let $G = (V, E_c, E_s)$ have separate directed compromise-reachability edges E_c and required-service edges E_s. A design installs $E'_c \subseteq E_c$ and separately declares E'_s; no implication between the two edge types is presumed. For each initial compromise node c, $\mathsf{Reach}_{(V,E'_c)}(c) \subseteq \mathsf{Reach}_{(V,E_c)}(c)$. If a checker also verifies the required service paths in (V, E'_s) with any declared capacities and timing conditions, then the*

design meets the conjunction of these separate modeled *containment and service obligations.*

Proof. Every directed compromise path using edges in E'_c is also a path in E_c. Independent certified service-path witnesses, and any separately assumed capacity/timing obligations, establish the other conjunct. No equivalence between compromise and service links is used. □

Corollary 8.2 (Dependence-guided segmentation)**.** *A D federation can seek minimal edge-cut guards whose deletion witnesses establish which barriers are critical, while $B^\star$ coordinates separate containment and service certificates. Such certificates do not validate the physical graph or all real-world failure modes.*

8.3 Contradiction, quarantine, and false security

A classical inconsistent context proves every formula, so a verifier checking only syntactic derivability does not by itself supply useful security assurance. The C federation has a special role: it can raise a certified inconsistency flag for the precise context in which its receipt is accepted.

Theorem 8.3 (Scoped quarantine and historical report revocation)**.** *Suppose the reporting policy Π_0 is enforced at every report transition; all safety displays require an explicit nonvacuous context-consistency receipt; and a checked C receipt for γ causes an atomic quarantine update covering γ and every certified extension scope. Then no* new *safety display for an affected scope is emitted after that update, and all previously published* protocol-governed *safety displays in affected scopes are marked* REVOKED *in the versioned report log. Unrelated scopes are not revoked by this receipt alone.*

Proof. The frozen report rule suppresses new safety displays for all scopes in the checked extension closure. Its atomic quarantine transition marks precisely the existing affected displays revoked; the extension predicate excludes unrelated scopes. The claim concerns the governed log, not ungoverned external copies. □

Remark 8.4. This rule prevents reporting from a *detected* inconsistent context. It cannot guarantee detection of every inconsistency, and finite inability to prove consistency is not proof of inconsistency.

8.4 Finite Ocean suites and universal claims

Proposition 8.5 (Finite-test nonuniversality)**.** *Passing a finite set O of sampled tests for a property over an infinite domain does not logically imply universal validity without an additional coverage or induction theorem.*

Proof. Let $O \subset \mathbb{N}$ be finite and choose $n \notin O$. Define $p(k)$ to be true exactly when $k \neq n$. Then all tests in O pass but $\forall k\, p(k)$ is false. □

9. Preregistered experiments and open problems

9.1 No execution is asserted

The architecture has been mathematically specified in this manuscript; there is no assertion that five FORGE-6 runtimes, $B^\star$, a cross-federation bus, or an independent checker have been installed or executed. Research must distinguish a paper specification, a simulated protocol, a checker trace and a successful independent reproduction.

Protocol 9.1 (Minimum frozen-run record)**.** For each experiment record *before starting*:

1. target T, definitions, complete Γ_0, environments, labels, optional guard family and typed checker V with version/hash;
2. frozen K_0, authorized provisional L_t, exact checker semantics, trusted computing base and failure policy;
3. all five role-local FORGE-6 instances, internal policies, seeds, $B^\star$ policy, legal service ticks, clock and spawn permissions;
4. initial bus state OFF and any authorized ON/OFF transitions, packet schema, cross-federation gateway, and full cost accounts;
5. budgets, stop rules, frozen Π_0, revocation and pending rules, and the six outcome categories, with UNKNOWN never relabeled as I or D;
6. preregistered baselines: legacy four-role, five-role AMLD, passive $B^\star$, five FORGE-6 without shared routing, and full controller with each ablation separately defined;
7. metrics: accepted settlements, complete scoped evidence, proof size, checks, time, total charged work, memory, duplication, missed dependencies, unauthorized promotion count, availability and replay receipts.

9.2 Exact questions worth testing

Conjecture 9.2 (Conditional dependence yield). On some guard-structured security families, an explicit D federation will discover more certified minimal-assumption atlases per charged resource than an otherwise matched architecture without D, after accounting for its extra cost.

Conjecture 9.3 (Controlled search frontier). On some repeated target families, policy-admissible $B^\star$ coordination improves the Pareto frontier of verified settlement, context fidelity, nonregression and total charged work over fixed scheduling. No unconditional claim is made.

Conjecture 9.4 (Strategic quotient discovery). There are structured security-game families in which verifier-certified, controller-respecting quotients reduce the number of distinct searched strategic states enough to outweigh quotient discovery and lifting costs.

Conjecture 9.5 (Five-federation proof-DAG reuse). For some target distributions, cross-role reuse of identical scope-compatible certified proof nodes exceeds all additional communication, verification and scheduling overhead. This is the empirical existence question suggested by Theorem 6.4.

9.3 Research boundary and conclusion

The theorems give a larger, carefully typed structure of conditional guarantees, not an unrestricted safety result. The system remains vulnerable to wrong specifications, incomplete environments, erroneous checkers, compromised execution authorities, insufficient budgets, unfair policies, unverifiable model claims, and mistaken scope transfers. In particular, no amount of schedule creativity proves T where the fixed theory does not support it.

The conceptual advance from *AMLD Security 3* is therefore a move from isolated, verifier-gated application sketches to a specified *investigative institution*: five complementary FORGE-6 federations search for differently typed conclusions, D accounts for assumptions, and $B^\star$ chooses legal investigative direction without deciding mathematical truth. What this institution can *discover* is an experimental question; what it may *certify* is an explicitly formal one.

A. Theorem audit and status register

ATLAS repair record (October 21, 2026). This edition directly responds to F01–F11 of the separately issued ATLAS-style v0.1 audit. It is a reasoned mathematical repair pass, not a claim that a separate autonomous ATLAS program or proof assistant verified these theorems. Critical changes include a nonvacuous guard poset, frozen scoped reporting, macrostep quotient semantics, wall-time unit separation and historical safety-report revocation. Concrete finite adversarial fixtures and a further audit are supplied in the accompanying repair/re-audit package.

Result	What is actually proved / what is not
Frozen arbitration	Deterministic scoped evidence vector, display ordering and revocation; not a logical priority of truths.
Typed cardinality	31 components by construction; not 31 deployed programs.
Federated noncontamination	Every ledger insertion checked; semantic truth additionally assumes sound V.
Verifier neutrality	Same complete raw verifier input, same result; ledger admission can depend on current state.
Guard dependence	Strong receipts imply base semantic nonentailment in both directions; typed I requires separate check.
Nonvacuous minimal-guard basis	Relative up-set inside satisfiable guards; complete atlas also needs checked SAT, negative and coverage evidence.
Dual-model bridge	Strong, nonvacuous dependence witnesses semantic nonentailment both ways at the base context; typed I requires a separate check.
Deletion-witness minimality	Verified immediate deletions certify inclusion-minimal guard.
Scope safety	No unlicensed dependency paths if gates enforced.
Passive embedding	Trace projection with sufficient extra budget; not free simulation.
Two-level cyclic bound	$25m$ executed dispatches; wall time only with uniform duration and final-check limits.
DAG reuse	Exact accounting identity; saving iff avoided duplication exceeds overhead.
No universal cost dominance	Fixed overhead defeats blanket strict speedup.
Game-aware quotient	Finite-horizon reachability under explicit complete one-service macrosteps; not elapsed time or general stochastic cost.

Result	What is actually proved / what is not
Inconsistency quarantine	Scoped contradiction blocks new safety reports and tags old governed ones revoked; not universal detection.
Infrastructure example	Graph-reachability and certified paths only; not physical safety.
Ocean counterexample	Finite suites are not universal proof without more premises.

B. Source and notation notes

$B^\star$	Distinguished global Player-0; not BANK and not an internal B_0^r.
B_0^r	Role-local FORGE-6 coordinator, without direct mathematical proof authority.
B_j^r	For $j = 1, \ldots, 5$, one of five role-local mathematical ATPs.
K_0	Frozen, certified BANK baseline. No silent write-back.
L_t	Provisional, verifier-accepted and support-closed ledger.
U_t	Speculative candidate store; not trusted assertions.
D	Dependence role, with strong guarded receipts as one explicit semantics.
I	Independence relative to a fixed theory and declared metatheory.
C	Contradiction/inconsistency relative to a declared context.
Π_0	Immutable scoped arbiter and revocation policy.
$\mathcal{C}$	Family of satisfiable guard contexts; inconsistent contexts are inadmissible for the security atlas.
$\overset{a}{\Rightarrow}$	One charged service with complete zero-service microstep closure.
UNKNOWN	Bounded nonsettlement after stop, not pending, independence, or a role.

References

Bibliography

[1] B. Tenneson, *AMLD Security 3: Eight Formal Application Sketches for Intelligent Target Settlement*, companion research manuscript v1.0, September 2, 2026. Primary source for the eight domain sketches and existing verification principles.

[2] B. Tenneson, *Security by Separation, Verification, and Settlement: An AMLD-Oriented Mathematical Treatise*, working research manuscript v0.1, August 30, 2026. As cited in *AMLD Security 3*.

[3] B. Tenneson, *What Checks the Proof? A Textbook Introduction to Automated Mathematical Logic Deciders*, v6.80, August 22, 2026. As cited in *AMLD Security 3*.

[4] B. Tenneson, *FORGE-6: Federated Orchestration of Reasoning, Graphs, and Exchange*, architectural description, October 6, 2026 (source note describing internal B_0 and mathematical ATPs B_1–B_5).

[5] B. Tenneson, *Depths of Induction*, version 62, chapter 13, "AMLD+0: Certified Strategy," October 2026 (describes $P_0 = B^\star$, passive delegation, nonstarvation and committed baseline-policy replay).

[6] J. Saltzer and M. Schroeder, "The Protection of Information in Computer Systems," *Proceedings of the IEEE*, 63(9), 1975, pp. 1278–1308.

Editorial and verification note. This illustrated monochrome sequel is a formal research draft prepared with AI assistance using the cited author-supplied manuscript and architectural notes. The numbered propositions and theorems have been checked for assumption/conclusion coherence in drafting but have not been independently formalized in a proof assistant or validated by a separate mathematical referee. The four explicitly labeled conjectures remain open proposals. The named author should review all new definitions, sources and publication claims before release.